

Curso Superior de Tecnologia para Internet

Disciplina: Gerenciamento de Dados para Web **Professor:**

Fernando Jorge Ebrahim Lima e Silva

Aluno: Ulisses da Silva Bembem

Matrícula: 20222010350198

PROVA SIMULADA: XML e APIs Relacionadas

Instruções:

- Leia atentamente cada questão.
- Responda de forma clara e objetiva.
- Utilize os conhecimentos adquiridos sobre XML, DOM, AJAX e APIs XML em Java (DOM, SAX, JAXB).

Seção 1: Compreensão e Manipulação de XML com JavaScript (DOM e AJAX)

Arquivo XML de Exemplo (crie um arquivo chamado filmes.xml):

XML

```
<filmes>
  <filme id="F001">
    <titulo>A Origem</titulo>
    <diretor>Christopher Nolan</diretor>
    <ano>2010</ano>
    <genero>Ficção Científica</genero>
  </filme>
  <filme id="F002">
    <titulo>O Poderoso Chefão</titulo>
    <diretor>Francis Ford Coppola</diretor>
    <ano>1972</ano>
    <genero>Drama</genero>
  </filme>
  <filme id="F003">
    <titulo>Interestelar</titulo>
    <diretor>Christopher Nolan</diretor>
    <ano>2014</ano>
    <genero>Ficção Científica</genero>
  </filme>
</filmes>
```

O código completo está disponível no github: <https://github.com/ubembem/GerenciamentoDadosParaWeb.git>

Questões:

1. Exibir Títulos dos Filmes (DOM): Crie um trecho de código JavaScript que, ao carregar o arquivo filmes.xml, acesse todos os elementos <titulo> e exiba o texto de cada título em uma lista não ordenada () dentro de um elemento div com id="lista-titulos-filmes" em sua página HTML.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Filmes</title>
</head>
<body>
<h1>Lista de filmes</h1>
<div id="lista-titulos-filmes"></div>
<script>
function loadXMLDoc() {
    const xhr = new XMLHttpRequest();
    xhr.open("GET", "filmes.xml", true);
    xhr.onreadystatechange = function () {
        if (xhr.readyState === 4 && xhr.status === 200) {
            const xmlDoc = xhr.responseXML;
            const titulos = xmlDoc.getElementsByTagName("titulo");
            const ul = document.createElement("ul");
            for (let i = 0; i < titulos.length; i++) {
                const li = document.createElement("li");
                li.textContent = titulos[i].textContent;
                ul.appendChild(li);
            }
            document.getElementById("lista-titulos-filmes").appendChild(ul);
        }
    };
    xhr.send();
}
window.onload = loadXMLDoc;
</script>
</body>
</html>
```

2. Informações de um Filme por Atributo (DOM): Como você acessaria o segundo elemento <filme> do arquivo filmes.xml e, utilizando o método

getAttribute(), obteria o valor do atributo id desse filme? Exiba o ID, título e ano desse filme em um div com id="filme-especifico".

Para acessar o segundo elemento <filme> do arquivo filmes.xml, fiz o seguinte: primeiro, usei um XMLHttpRequest para carregar o arquivo filmes.xml com uma requisição GET assíncrona. Quando a requisição terminava com sucesso, pegava o documento XML com responseXML e usava getElementsByTagName("filme") para acessar a lista de filmes. Como precisava do segundo filme, selecionei o elemento na posição 1. Depois, usei getAttribute("id") para pegar o valor do atributo id e getElementsByTagName para extrair o texto dos elementos <título> e <ano>. Por fim, juntei essas informações em uma string e a coloquei no innerHTML do div com o id="filme-especifico", mostrando algo como "ID: F002, Título: O Poderoso Chefão, Ano: 1972".

3. Acessando o Primeiro Diretor (DOM - firstChild, nodeValue): Utilizando firstChild e nodeValue, explique como você extrairia o nome do diretor do primeiro filme do filmes.xml e o exibiria em um div com id="primeiro-diretor".

Para extrair o nome do diretor do primeiro filme, usamos firstChild para acessar o nó de texto dentro do elemento <diretor> e nodeValue para obter seu valor.

4. Adicionando um Novo Filme (DOM - Manipulação): Descreva os passos para adicionar um novo elemento <filme> completo (com título, diretor, ano e gênero) ao seu objeto DOM em memória, representando-o no XML filmes.xml.

Para adicionar um novo elemento <filme> ao objeto DOM do arquivo filmes.xml, comecei criando um novo elemento <filme> com createElement. Depois, adicionei o atributo id usando setAttribute para dar um identificador único, como "F004". Em seguida, criei os elementos filhos <título>, <diretor>, <ano> e <genero> com createElement e adicionei valores de texto a cada um usando createTextNode, como "Matrix" para o título, "Wachowski" para o diretor, "1999" para o ano e "Ficção Científica" para o gênero. Anexei esses elementos ao <filme> com appendChild e, por fim, adicionei o novo <filme> ao elemento raiz <filmes> do documento XML, também com appendChild, garantindo que ele fosse incluído na estrutura do DOM em memória.

5. Removendo um Filme (DOM - Manipulação): Explique como você removeria o filme com id="F002" (O Poderoso Chefão) do seu objeto DOM em memória.

Para remover o filme com id="F002" do objeto DOM, primeiro peguei todos os elementos <filme> com getElementsByTagName. Depois, percorri essa lista com um laço e, para cada filme, verifiquei o valor do atributo id usando getAttribute. Quando encontrei o filme com id="F002", usei o método parentNode.removeChild para

removê-lo do DOM, passando o próprio elemento como argumento. Para garantir que o laço parasse após a remoção, adicionei um break, já que o filme com esse id é único.

6. Encontrando Filmes de um Diretor Específico (DOM): Como você percorreria o XML para encontrar e exibir os títulos de todos os filmes dirigidos por "Christopher Nolan" em um div com id="filmes-nolan"?

Para exibir os títulos dos filmes dirigidos por "Christopher Nolan" em um div com id="filmes-nolan", comecei carregando o arquivo filmes.xml com um XMLHttpRequest em uma requisição GET assíncrona. Quando a requisição terminava com sucesso, pegava o documento XML com responseXML e obtinha todos os elementos <filme> usando getElementsByTagName. Depois, percorri cada filme, verificando o texto do elemento <diretor> com textContent. Se o diretor fosse "Christopher Nolan", criava um elemento com o título do filme, também obtido com textContent, e adicionava esse a uma lista não ordenada . Por fim, anexe a lista ao div com id="filmes-nolan" usando appendChild, exibindo os títulos dos filmes correspondentes.

7. Propriedades do XMLHttpRequest (AJAX - Múltipla Escolha): Qual propriedade do objeto XMLHttpRequest armazena o status da requisição (por exemplo, 0, 1, 2, 3, 4)?

- ☐ a) responseText
- ☐ b) responseXML
- ☒ c) readyState
- ☐ d) status

8. Requisição GET com Tratamento de Resposta (AJAX - Prática): Crie um código JavaScript que realize uma requisição GET para um arquivo chamado "dados.txt" e exiba o conteúdo desse arquivo em um elemento HTML com o id="resultado". Inclua o tratamento da propriedade onreadystatechange para garantir que a resposta seja exibida apenas quando a requisição for concluída com sucesso (status 200).

```
function loadText() {  
    const xhr = new XMLHttpRequest();  
    xhr.open("GET", "dados.txt", true);  
    xhr.onreadystatechange = function () {  
        if (xhr.readyState === 4 && xhr.status === 200) {  
            document.getElementById("resultado").textContent = xhr.responseText;  
        }  
    }  
}
```

```

    } else if (xhr.readyState === 4) {
        document.getElementById("resultado").textContent = "Erro ao carregar dados.";
    }
};
xhr.send();
}

window.onload = loadText;

```

Seção 2: APIs XML em Java (DOM, SAX, JAXB)

Questões:

9. API de Árvore de Nós (Múltipla Escolha): Qual das seguintes APIs para XML decompõe um documento XML em uma árvore de nós, permitindo navegação e manipulação?

- ☐ a) SAX
- ☐ b) JAXB
- ☒ c) DOM
- ☐ d) JAXP

10. Anotação JAXB para Elemento Raiz (Múltipla Escolha): Em JAXB, qual anotação é usada para mapear a classe de mais alto nível para um elemento raiz no XML?

- ☐ a) @XmlElement
- ☒ b) @XmlRootElement
- ☐ c) @XmlType
- ☐ d) @XmlAccessorType

11. Tipo de Nó na Interface Node (Múltipla Escolha): Qual método da interface Node (no DOM) retorna o tipo de nó (por exemplo, Node.ELEMENT_NODE, Node.TEXT_NODE)?

- ☐ a) getNodeName()
- ☐ b) getNodeValue()
- ☒ c) getNodeType()
- ☐ d) getParentNode()

12. Processo de Marshalling em JAXB (Múltipla Escolha): No contexto do JAXB, o processo de converter objetos Java em um documento XML é chamado de:

- ☐ a) Bind
- ☐ b) Unmarshal
- ☐ c) Parse
- ☒ d) Marshal

13. Diferença SAX vs. DOM (Aberta): Descreva a diferença fundamental entre as APIs SAX e DOM para interpretar documentos XML, considerando seus modelos de processamento.

A principal diferença entre SAX e DOM está no jeito que eles lidam com o XML. O SAX é como se fosse uma leitura linha por linha: ele vai passando pelo arquivo XML e dispara eventos toda vez que encontra um elemento, atributo ou texto. Isso é ótimo para arquivos grandes porque usa pouca memória, mas é mais complicado se você quer voltar ou manipular a estrutura, já que ele só vai pra frente. Já o DOM carrega o XML inteiro na memória, criando uma árvore com todos os nós. Isso facilita muito navegar e alterar o documento, mas se o arquivo for grande, pode pesar bastante na memória. Então, SAX é mais leve e rápido para leitura simples, enquanto DOM é melhor para manipulações mais complexas.

14. Processo de Serialização DOM (Aberta): Discorra sobre o processo de serialização de uma árvore DOM (em Java), mencionando as classes e métodos principais envolvidos para converter um objeto Document de volta para uma representação XML textual.

Serializar uma árvore DOM é basicamente transformar aquele objeto Document que tá na memória em um texto XML, tipo salvar num arquivo ou mostrar na tela. Pra fazer isso em Java, eu uso algumas classes principais. Primeiro, pego uma TransformerFactory com TransformerFactory.newInstance() pra criar um Transformer. Esse Transformer é quem faz o trabalho pesado, usando o método transform. Eu passo o Document como entrada através de um DOMSource e defino pra onde o XML vai, como um arquivo ou o console, com um StreamResult. Também dá pra configurar coisas como indentação com setOutputProperty, pra deixar o XML mais legível. Por exemplo, pra salvar no console, eu faria algo como transformer.transform(new DOMSource(doc), new StreamResult(System.out)). É bem direto, mas precisa configurar certinho pra sair como esperado.

15. Criação de Classe JAXB e Marshalling (Prática em Java): Considerando o JAXB, crie uma classe Java simples chamada Item com os atributos nome (String) e preco (double). Adicione as anotações JAXB necessárias para que esta classe

possa ser mapeada para um elemento XML chamado <item> com os sub-elementos <nome> e <preco>. Demonstre como você realizaria o processo de marshalling de um objeto Item para a saída padrão (System.out).

Para criar uma classe simples chamada Item com os atributos nome e preco, configurada para o JAXB gerar um XML com a estrutura pedida. Depois, mostro como transformar um objeto dessa classe num XML e exibir no console.

Nessa resposta, criei a classe Item com as anotações @XmlRootElement para definir o elemento raiz <item> e @XmlElement pros campos nome e preco. No método main, usei o JAXB para transformar o objeto num XML e jogar na saída padrão, com uma formatação legal para ficar fácil de ler. Se der algum erro, mostro uma mensagem simples no console.

Implementação:

```
import javax.xml.bind.JAXBContext;
import javax.xml.bind.Marshaller;
import javax.xml.bind.annotation.XmlElement;
import javax.xml.bind.annotation.XmlRootElement;
```

```
@XmlRootElement(name = "item")
```

```
public class Item {
    private String nome;
    private double preco;
```

```
    // Construtor vazio pro JAXB
    public Item() {}
```

```
    // Construtor com parâmetros
    public Item(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
    }
```

```
    @XmlElement
    public String getNome() {
        return nome;
    }
```

```
    public void setNome(String nome) {
        this.nome = nome;
    }
```

```
    @XmlElement
    public double getPreco() {
        return preco;
    }
```

```

public void setPreco(double preco) {
    this.preco = preco;
}

public static void main(String[] args) {
    try {
        // Criando um objeto Item
        Item item = new Item("Caderno", 15.99);

        // Configurando o JAXB pra transformar em XML
        JAXBContext context = JAXBContext.newInstance(Item.class);
        Marshaller marshaller = context.createMarshaller();

        // Pra deixar o XML bonitinho, com quebras de linha
        marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);

        // Exibindo no console
        marshaller.marshal(item, System.out);
    } catch (Exception e) {
        System.out.println("Deu erro ao gerar o XML: " + e.getMessage());
    }
}
}

```